

Doc link:-

<https://docs.google.com/document/d/1MWuPgoy9c87mX9WlnZlqvsioC8pWfEpPzNtXfKUfpys/e/dit?usp=sharing>

Chapter link:- <https://jax-ml.github.io/scaling-book/profiling/>

Question 1: take a look at [this](#) Colab/profile and figure out what looks suspicious and what's going on here. Can you tell me exactly what computations are happening and what each operation is doing? What are the true shapes of each matrix involved and how are they sharded? *Try looking at the profile first without reading the code.*

My answer:-

there are 2 really big mat muls fusion of size

```
bf16[8192] * bf16[4096, 8192] -> bf16[4096]
```

Per shard

Question 2: [The Transformer Colab](#) from earlier

https://colab.research.google.com/drive/1_6krERgtolH7hbUlo7ewAMLlbA4fqEF8?usp=sharing

implements a simple mock Transformer. Follow the instructions in the Colab and get a benchmark of the naive Transformer with GSPMD partitioning. How long does each part take? How long should it take? What sharding is being used? Try fixing the sharding! *Hint: use `jax.lax.with_sharding_constraint` to constrain the behavior. With this fix, what's the best MXU you can get?*

For reference, the initial version gets roughly 184ms / layer and the optimized profile gets 67 ms / layer. Once you've done this, try staring at the profile and see if you can answer these questions purely from the profile:

- What sharding strategy is this?
- What is the batch size, , ?
- What fraction of time is spent on attention vs. the MLP block?
- What fraction of time should be spent on each op at the roofline?

Note: since this problem was written, the XLA compiler has gotten better. The initial version is now at roughly 90ms / layer and the optimized profile is only about 10ms / layer better (80 ms / layer). Still, it's worth playing with and seeing if you can do better.

My Answer:-

ShardingTypeError: Use ``...`.get(out_sharding=)`` to provide output PartitionSpec for the gather indexing as out sharding could not be resolved unambiguously (or would require collectives on inputs). Got operand=ShapedArray(bfloat16[32768,8192]), indices=ShapedArray(int32[8@data,1024,1])

1st Normal matrix multiplication

```
x = jax.random.normal(key, (32, 1024, 8192)).astype(jnp.bfloat16)
W = jax.random.normal(key, (8192, 8192 * 4)).astype(jnp.bfloat16)
y = x @ W
```

only on 1 TPU so we have operations=32*2*1024*8192*8192*4FLOPS=17.6 TFLops

TPU= v5e so the time taken should be=17.6/197=89ms

real time taken is [94ms 132µs 883ns](#)
so we are close

Name	fusion
Category	N/A
Start time	00:00:10.230 867 503
✓ Duration	94ms 132µs 883ns
Thread	XLA Ops [3]
Process	/device:TPU:0 [3]
SQL ID	slice[165046] ▾

3rd operation

Pure data parralelism for

```
sharding = jax.sharding.NamedSharding(mesh, jax.sharding.PartitionSpec('data', None, None))
x = jax.lax.with_sharding_constraint(x, sharding)
# [B, T, D] -> [B, T, F] -> [B, T, D]
x = jnp.dot(jax.nn.relu(jnp.dot(x, w1)), w2)
# [B, T, D] -> [B, T, V]
return jax.nn.log_softmax(jnp.dot(x, embed.T))
```

for

bs = 8

```
seqlen = 1024
vocab_size = 32_768
d_model = 8_192
d_ff = 16_384
```

```
tokens = jnp.zeros((bs, seqlen), dtype=jnp.int32) # tokens of shape (batch_size, seq_len)
embed = jnp.zeros((vocab_size, d_model), jnp.bfloat16) # a single weight matrix of shape
(d_model, 2 * d_model)
w1 = jnp.zeros((d_model, d_ff), jnp.bfloat16) # a single weight matrix of shape (d_model, 2 *
d_model)
w2 = jnp.zeros((d_ff, d_model), jnp.bfloat16) # a single weight matrix of shape (d_model, 2 *
d_model)
```

so the time

$\text{jnp.dot}(x, w1)$ is 2BTDF=275e9 FLOPS per device=1.4 ms= in the graph at [1ms 430μs 841ns](#)

same for [BTF]->BTD

BTD->BTV will take $1.4\text{ms} \times 32768 / 16384 = 2.8\text{ms}$ = [2ms 910μs 179ns](#)

As predicted

2rd Operation no parallelism takes 8x time of 3rd operation

4th FSDP

```
tokens = jax.device_put(tokens, P('data', None)) # sharded along the
batch dimension

w1 = jax.device_put(w1, P('data', None))
w2 = jax.device_put(w2, P(None, 'data'))
embed = jax.device_put(embed, P(None, None))
```

should take same time as data parallelism considering

WICI is too much so its not worth it

it takes 6 +6ms just to 2 communication involved in FSDP

same for 5th operation tensor parallelism

it is Wici bound

originally it was doing all reduce when logits

so if we remove it then only reuce scatter which again takes 6 secs

Running LLM

time taken by whole function= 216ms 168µs 161ns

after a bit of decoding its running in correct time 12ms 251µs 354ns per matrix multiplication

Kaggle notebook for optimised and debugged implementation of this part 2:-

<https://www.kaggle.com/code/reptor420/chapter-9-problem-2-jax-scaling-book>